

A Flexible-Granularity Task Graph Representation and Its Generation from C Applications (WIP)

Tiago Santos
INESC-TEC, FEUP
Universidade do Porto
Porto, Portugal
tiagolascasas@fe.up.pt

João Bispo
INESC-TEC, FEUP
Universidade do Porto
Porto, Portugal
jbispo@fe.up.pt

João M. P. Cardoso
INESC-TEC, FEUP
Universidade do Porto
Porto, Portugal
jmpc@fe.up.pt

Abstract—Modern hardware accelerators, such as FPGAs, allow offloading large regions of C/C++ code in order to improve the execution time and/or the energy consumption of software applications. An outstanding challenge with this approach, however, is solving the Hardware/Software (Hw/Sw) partitioning problem. Given the increasing complexity of both the accelerators and the potential code regions, one needs to adopt a holistic approach when selecting an offloading region by exploring the interplay between communication costs, data usage patterns, and target-specific optimizations. To this end, we propose representing a C application as an extended task graph (ETG) with flexible granularity, which can be manipulated through the merging and splitting of tasks. This approach involves generating a task graph overlay on the program’s Abstract Syntax Tree (AST) that maps tasks to functions and the flexible granularity operations onto inlining/outlining operations. This maintains the integrity and readability of the original source code, which is paramount for targeting different accelerators and enabling code optimizations, while allowing the offloading of code regions of arbitrary complexity based on the data patterns of their tasks. To evaluate the ETG representation and its compiler, we use the latter to generate ETGs for the programs in Rosetta and MachSuite benchmark suites, and extract several metrics regarding data communication, task-level parallelism, and dataflow patterns between pairs of tasks. These metrics provide important information that can be used by Hw/Sw partitioning methods.

Index Terms—Task Graph, Source-to-Source Compiler, Hardware/Software Partitioning, Hardware Accelerators, FPGA

PROJECT BACKGROUND

Modern hardware accelerators, such as FPGAs, allow offloading large regions of C/C++ code in order to improve the execution time and/or the energy consumption of software applications. This approach often involves two processes: hardware/software partitioning, which identifies regions to offload to the FPGA; and optimizing those regions (e.g., through HLS directives). As both processes are separate and usually applied in sequence, the interplay between them is unnatural, and it is unclear how the choices made in one step can benefit the choices made in the other step. Therefore, as FPGA-based accelerators increase in complexity, creating one unified process of partitioning and optimization becomes of paramount importance. Therefore, we propose a unified process as a holistic approach to partitioning and optimization:

Santos was supported by a Fundação para a Ciência e Tecnologia (FCT) PhD scholarship (DOI 10.54499/2021.07324.BD).

by performing a single pass over the program’s task graph, we want to select a code region for offloading by exploring the interplay between communication costs, data usage patterns, and target-specific optimizations [1].

To this end, we present a paper [2] that establishes one important step towards achieving our proposed holistic approach: the definition of the Extended Task Graph (ETG), a bespoke intermediate representation of a program’s source code that exposes the information required for this type of analysis, as well as enabling the offloading of arbitrarily-complex code regions while preserving the source code’s semantics and readability. Fig. 1 shows an example of an ETG.

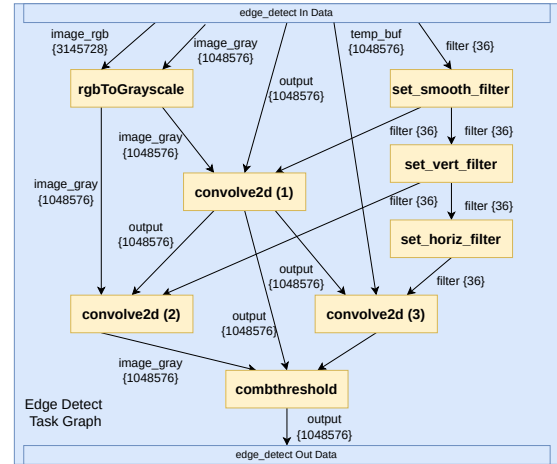


Fig. 1. Example of an ETG for an edge detection application. Edges represent the data communicated between tasks, as well as their size in bytes

REFERENCES

- [1] T. Santos, J. Bispo, and J. M. Cardoso, “A cpu-fpga holistic source-to-source compilation approach for partitioning and optimizing c/c++ applications,” in *2023 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2023, pp. 320–322.
- [2] T. Santos, J. Bispo, and J. M. P. Cardoso, “A flexible-granularity task graph representation and its generation from c applications (wip),” in *Proceedings of the 25th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES '24)*. New York, NY, USA: ACM, June 2024, p. 5. [Online]. Available: <https://doi.org/10.1145/3652032.3657580>